

UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA  
FACULTAD DE INGENIERÍA ARQUITECTURA Y DISEÑO

INGENIERÍA EN SOFTWARE Y TECNOLOGÍAS

EMERGENTES



Paradigmas de la programación

## TALLER 3

ADRIAN BALDERAS ROSAS 373488

Carlos Gallegos

## EJERCICIO 1

```
#include <stdio.h>

int global_var = 10;

void func() {
    int local_var = 20;
    static int static_var = 30;
    local_var++;
    static_var++;
    printf("local_var: %d, static_var: %d\n", local_var, static_var);
}

int main() {
    func();
    func();
    return 0;
}
```

Salida del programa

Primera llamada a func()

local\_var = 21, static\_var = 31

Segunda llamada a func()

local\_var = 21, static\_var = 32

la salida es:

local\_var: 21, static\_var: 31

local\_var: 21, static\_var: 32

Tipos de memoria usados

global\_var: memoria estática.

local\_var: memoria automática (stack).

static\_var: memoria estática (aunque esté en una función).

No se usa memoria dinámica (heap) porque no hay malloc/free.

## EJERCICIO 2

```
#include <stdio.h>
#include <stdlib.h>

void allocate_memory() {
    int *arr = (int *)malloc(5 * sizeof(int));
    for (int i = 0; i < 5; i++) {
        arr[i] = i * 2;
    }
    printf("arr[2]: %d\n", arr[2]);
}

int main() {
    allocate_memory();
    return 0;
}
```

1. ¿Qué problema de manejo de memoria presenta este código?

El problema principal es una fuga de memoria

2. ¿Cómo podríamos solucionarlo?

Debemos liberar explícitamente la memoria que reservamos, usando la función `free()` justo antes de que ya no la necesitemos. En este caso, el lugar correcto es al final de la función `allocate_memory`, después de haber usado el arreglo.

### EJERCICIO 3

```
#include <stdio.h>
#include <pthread.h>

int shared_var = 5; // Variable compartida entre hilos

void *modify_variable(void *arg) {
    shared_var += 10;
    return NULL;
}

int main() {
    pthread_t thread;
    pthread_create(&thread, NULL, modify_variable, NULL);
    pthread_join(thread, NULL);
    printf("shared_var: %d\n", shared_var);
    return 0;
}
```

1. ¿Cuál es la salida esperada del programa?

La salida esperada del programa es:

shared\_var: 15

2. ¿Qué pasaría si removemos pthread\_join(thread, NULL);?

Si eliminamos la línea pthread\_join(thread, NULL);, el comportamiento del programa se vuelve impredecible, Sin pthread\_join, no hay garantía sobre el orden de ejecución. La salida podría ser 5 o 15 cada vez que ejecutes el programa